# Day 162 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 24, Day 2 reference

Watch: Hypothesis Testing and The Null Hypothesis, Clearly Explained!!! (StatQuest with Josh Starmer) | Power Analysis, Clearly Explained!!! (StatQuest with Josh Starmer)

`day162_hypothesis_testing_ab.py`, segment by segment, in the order `main()` uses things. The parts (load_hillstrom, p_two_sided, two_prop_test, welch_test, permutation_test, n_per_arm, power_two_prop, mcnemar) are copied unchanged into `day162_exercise.py`; Day 161's Adult data and logistic regression and Day 109's trees and boosting are imported from their exercise files. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

**MATH BOX LEGEND**

**Symbols:** x, n = successes and group size (1 = emailed, 0 = control) · z, t = distance from 0 in spreads · p = a p value (or a rate, by context) · R = shuffles · e1, e2 = two models' mistakes (True = wrong) · b, c = rows only one model gets wrong

## Segment 0 — Setup

### Imports: the standard library's normal distribution, and earlier days' parts

**CODE**

```
from statistics import NormalDist
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day109_exercise as d109
    import day161_exercise as d161
```

- `from statistics import NormalDist` — Python's own normal distribution: `NormalDist().inv_cdf(0.975)` is 1.96.
- `sys.path.insert(0, HERE)` — Day 161's exercise file sits in this same folder.
- `import day161_exercise as d161` — the Adult data, Week 24's split and Newton logistic regression, unchanged.

## Segment 1 — The parts

### load_hillstrom: a CSV with quoted fields

**CODE**

```
def load_hillstrom():
    os.makedirs(DATA, exist_ok=True)
    path = os.path.join(DATA, "hillstrom.csv")
    if not os.path.exists(path):
        urllib.request.urlretrieve(HILL_URL, path)
    rows = list(csv.reader(open(path, encoding="utf-8", newline="")))
    d = {c: np.array([r[k] for r in rows[1:]]) for k, c in enumerate(rows[0])}
    for c in ("recency", "history", "mens", "womens", "newbie", "visit", "conversion", "spend"):
        d[c] = d[c].astype(float)
    return d
```

- `csv.reader(open(path, encoding="utf-8", newline=""))` — splits at commas, except inside quotes: "$750 - $1,000" stays one field.
- `rows[0]` — the first line is the header: the column names.
- `d[c].astype(float)` — text to numbers, for the columns that hold numbers.

## p_two_sided and two_prop_test

**MATH**

$$z = (p_1 - p_0) / \sqrt{(p_{pool}(1-p_{pool})(1/n_1 + 1/n_0))}; \quad p = erfc(|z|/\sqrt{2})$$

**CODE**

```
def p_two_sided(z):
    return math.erfc(abs(z) / math.sqrt(2))
def two_prop_test(x1, n1, x0, n0):
    p1, p0 = x1 / n1, x0 / n0
    pp = (x1 + x0) / (n1 + n0)
    se0 = math.sqrt(pp * (1 - pp) * (1 / n1 + 1 / n0))
    z = (p1 - p0) / se0 if se0 > 0 else 0.0
    se = math.sqrt(p1 * (1 - p1) / n1 + p0 * (1 - p0) / n0)
    return p1 - p0, z, p_two_sided(z), p1 - p0 - Z975 * se, p1 - p0 + Z975 * se
```

- `math.erfc(abs(z) / math.sqrt(2))` — both normal tails beyond |z|; erfc stays accurate where 1 – cdf would round to 0.
- `pp = (x1 + x0) / (n1 + n0)` — the pooled rate: what both groups would share if there were no effect.
- `z = (p1 - p0) / se0 if se0 > 0 else 0.0` — a conditional expression guards against dividing by 0 (no successes at all).
- `return p1 - p0, z, ..., p1 - p0 - Z975 * se, ...` — several values returned at once, as a tuple.

## welch_test and permutation_test

**CODE**

```
def welch_test(a, b):
    se = math.sqrt(a.var(ddof=1) / len(a) + b.var(ddof=1) / len(b))
    t = (a.mean() - b.mean()) / se
    return a.mean() - b.mean(), t, p_two_sided(t), a.mean() - b.mean() - Z975 * se, a.mean() -
        b.mean() + Z975 * se
def permutation_test(a, b, R=R_PERM, seed=0):
    x = np.r_[a, b]; obs = a.mean() - b.mean()
    rng = np.random.RandomState(seed); hits = 0
    for _ in range(R):
        p = rng.permutation(len(x))
        hits += abs(x[p[:len(a)]].mean() - x[p[len(a):]].mean()) >= abs(obs) - 1e-12
    return obs, (hits + 1) / (R + 1)
```

- `a.var(ddof=1)` — the sample variance, dividing by n – 1.
- `np.r_[a, b]` — joins the two groups end to end.
- `hits += abs(...) >= abs(obs) - 1e-12` — True counts as 1; the tiny margin keeps ties from being lost to rounding.
- `(hits + 1) / (R + 1)` — the real labelling counts as one of the possible orders, so p is never exactly 0.

### n_per_arm and power_two_prop

**MATH**

$$n = [z_{1-\alpha/2}\sqrt{(2p_{avg}(1-p_{avg})} + z_{power}\sqrt{(p_1(1-p_1) + p_0(1-p_0))}]^2 / (p_1 - p_0)^2$$

**CODE**

```
def n_per_arm(p1, p0, alpha=ALPHA, power=0.8):
    za, zb = NormalDist().inv_cdf(1 - alpha / 2), NormalDist().inv_cdf(power)
    pb = (p1 + p0) / 2
    return (za * math.sqrt(2 * pb * (1 - pb)) + zb * math.sqrt(p1 * (1 - p1) + p0 * (1 - p0))) **
        2 / (p1 - p0) ** 2
def power_two_prop(n, p1, p0, alpha=ALPHA):
    za = NormalDist().inv_cdf(1 - alpha / 2); pb = (p1 + p0) / 2
    se0 = math.sqrt(2 * pb * (1 - pb) / n); se1 = math.sqrt((p1 * (1 - p1) + p0 * (1 - p0)) / n)
    return NormalDist().cdf((abs(p1 - p0) - za * se0) / se1)
```

- `NormalDist().inv_cdf(1 - alpha / 2)` — the cut-off: 1.96 for $\alpha = 0.05$.
- `NormalDist().inv_cdf(power)` — 0.84 for 80% power.
- `NormalDist().cdf((abs(p1 - p0) - za * se0) / se1)` — the chance the observed difference lands beyond the cut-off.

### mcnemar: an exact binomial tail in log space

**MATH**

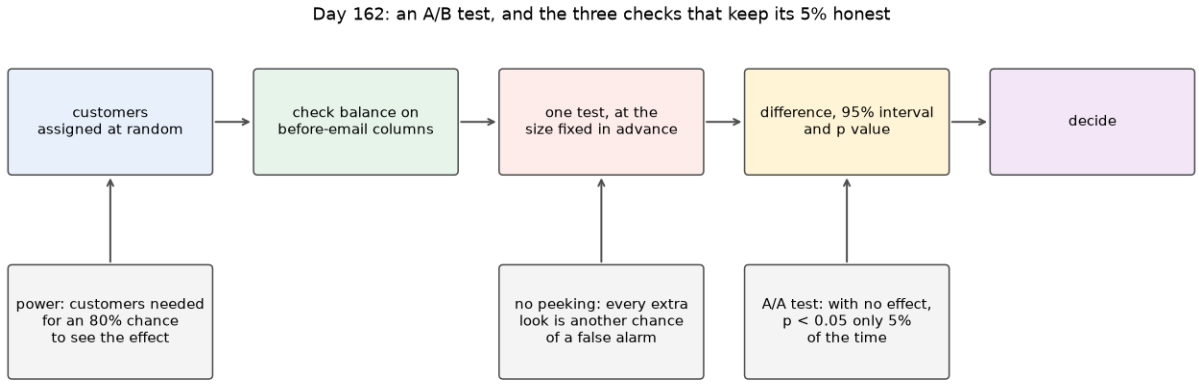
$$\chi^2 = (|b - c| - 1)^2 / (b + c); \quad exact\ p = 2\,\Sigma_{i \le min(b,c)}\, C(b+c, i) / 2^{b+c}$$

**CODE**

```
def mcnemar(e1, e2):
    b = int(np.sum(e1 & ~e2)); c = int(np.sum(~e1 & e2))
    chi2 = (abs(b - c) - 1) ** 2 / (b + c) if b + c else 0.0
    n, k = b + c, min(b, c)
    lp = [math.lgamma(n + 1) - math.lgamma(i + 1) - math.lgamma(n - i + 1) - n * math.log(2) for i
        in range(k + 1)]
    m = max(lp) if lp else 0.0
    exact = min(1.0, 2 * math.exp(m) * sum(math.exp(v - m) for v in lp)) if n else 1.0
    return b, c, chi2, math.erfc(math.sqrt(chi2 / 2)), exact
```

- `np.sum(e1 & ~e2)` — & is 'and', ~ is 'not' on boolean arrays: rows only model 1 gets wrong.
- `math.lgamma(n + 1) - math.lgamma(i + 1) - ...` — log of n!/(i!(n–i)!) · $2^{-n}$: no huge factorials, no overflow.
- `2 * math.exp(m) * sum(math.exp(v - m) for v in lp)` — the largest term factored out before exponentiating, so nothing underflows.
- `math.erfc(math.sqrt(chi2 / 2))` — a chi-square with 1 degree of freedom is a squared normal, so its tail is erfc.

SYSTEM DIAGRAM




*The A/B test pipeline and its three safeguards.*

## Segment 2 — The experiments

### demo_data: a balance test per before-email column

CODE

```
for a in ARMS[:2]:
    for x in cov.values():
        ps.append(welch_test(x[g[a]], x[c0])[2])
        smd.append((x[g[a]].mean() - x[c0].mean()) / math.sqrt((x[g[a]].var() + x[c0].var()) / 2))
```

- `for x in cov.values():` — loops over the dictionary's values: the columns themselves.
- `welch_test(x[g[a]], x[c0])[2]` — [2] picks the p value from the returned tuple.
- `math.sqrt((x[g[a]].var() + x[c0].var()) / 2)` — the standardised difference divides by the two groups' average spread.

### demo_aa: A/A splits and peeking with running totals

CODE

```
for s in range(S_AA):
    p = rng.permutation(c0); A, B = p[:h], p[h:2 * h]
    for col in ("visit", "conversion"):
        P[col].append(two_prop_test(d[col][A].sum(), h, d[col][B].sum(), h)[2])
        ca, cb = np.cumsum(d[col][A]), np.cumsum(d[col][B])
        sig = [two_prop_test(ca[n - 1], n, cb[n - 1], n)[2] < ALPHA for n in looks]
        ever[col] += np.maximum.accumulate(sig)
```

- `p = rng.permutation(c0); A, B = p[:h], p[h:2 * h]` — a random order of the untreated rows, cut into two halves.
- `np.cumsum(d[col][A])` — running totals: element n – 1 is the count after n customers.
- `[two_prop_test(ca[n - 1], n, cb[n - 1], n)[2] < ALPHA for n in looks]` — a test at every look, as a list of True/False.
- `np.maximum.accumulate(sig)` — running 'any so far': once a look is significant, every later entry stays True.

## demo_power: many simulated experiments at once

**CODE**

```
x1 = d[col][g[a]][rng.randint(0, len(g[a]), (R_POW, n))].sum(1)
x0 = d[col][c0][rng.randint(0, len(c0), (R_POW, n))].sum(1)
sim = np.mean([two_prop_test(u, n, v, n)[2] < ALPHA for u, v in zip(x1, x0)])
```

- `rng.randint(0, len(g[a]), (R_POW, n))` — a 1000 × n table of random row numbers: sampling with replacement.
- `[...].sum(1)` — the successes in each of the 1,000 experiments.
- `zip(x1, x0)` — pairs each emailed experiment with a control experiment.

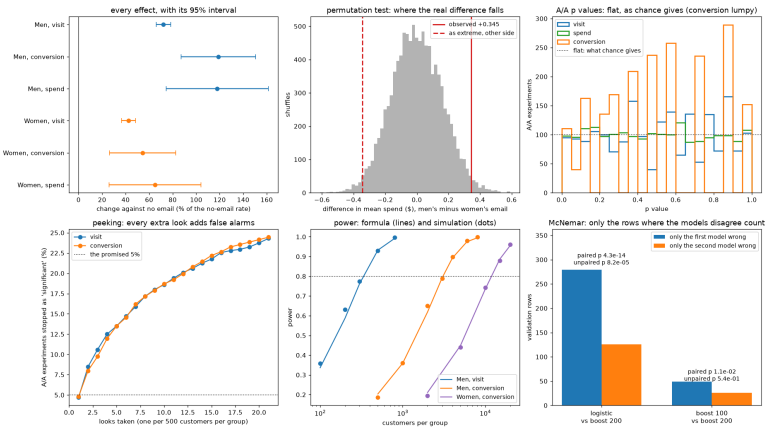
## demo_mcnemar: the same-procedure control

**CODE**

```
for s in range(S_SAME):
    p = np.random.RandomState(100 + s).permutation(fi); h = len(fi) // 2
    e = []
    for idx in (np.sort(p[:h]), np.sort(p[h:])):
        if name == "logistic":
            wh, _ = d161.fit_logistic(Xo[idx], y[idx]); pr = d161.predict_logistic(wh, Xo[va])
        else:
            t = d109.grow_reg(Xt[idx], y[idx].astype(float), np.arange(len(idx)), 64)
            d109.set_leaves(t, lambda i: y[idx][i].mean()); pr = d109.predict_reg(t, Xt[va])
        e.append((pr > 0.5) != yv)
    ps.append(mcnemar(e[0], e[1])[3])
```

- `for idx in (np.sort(p[:h]), np.sort(p[h:])):` — the two halves of one random cut, each training one model.
- `lambda i: y[idx][i].mean()` — each leaf's value: the share above $50K among its training rows (i indexes within idx).
- `e.append((pr > 0.5) != yv)` — True where the model is wrong.

**REAL GRAPH**




*Real graph from the script.*

## Quick Syntax Glossary — new or reinforced constructs

- `csv.reader(f)` — split lines at commas, respecting quotes.
- `NormalDist().inv_cdf(q)` — a normal quantile, from the standard library.
- `math.erfc(x)` — complementary error function: accurate normal tails.
- `math.lgamma(n + 1)` — log(n!), without computing n!.
- `a & ~b, np.maximum.accumulate(a)` — boolean and / not; a running maximum (running 'any' for booleans).

*End of Day 162 syntax reference. For the experiments and what they showed, see the Day 162 complete guide.*